Ziele dieser Semesterwoche 5

- Sie können die Unterschiede zwischen den Begriffen *Set*, *Map*, *Multiset* und *Multimap* erklären.
- Sie können eine *Map* implementieren.
- Sie verstehen die verschiedenen Varianten einer *Hash-Tabelle* und können unterschiedliche *Sondierungsverfahren* anwenden.

Maps

Eine Map modelliert eine *durchsuchbare Collection* von Schlüssel-Wert *(Key-Value) Entries*.

Hauptoperationen für eine Map sind:

- *Einfügen*
- *Suchen*
- *Löschen*

Pro Key ist ***nur eine*** *Entry* erlaubt !

Anwendungen:

- Einfache Datenbanken,
  z.B. Personalverzeichnis (Key = Personalnummer),
  Ortschaftenverzeichnis (Key = PLZ)

Map ADT

Map ADT Methoden:

- ***get(k):*** falls in der Map M eine Entry mit dem Schlüssel *k* vorkommt, wird der dazugehörige *Wert* zurückgeliefert; sonst *null.*
- ***put(k, v):*** falls in der Map M *keine* Entry mit dem Schlüssel *k* vorkommt, wird eine neue Entry(k, v) hinzugefügt und *null* zurückgegeben; andernfalls wird der vorhandene *Wert* durch *v* ersetzt und der *alte Wert* zurückgeben.
- ***remove(k):*** falls in der Map M eine Entry zum Schlüssel *k* vorkommt, dann wird er aus der Map M entfernt und der dazugehörige *Wert* zurückgegeben;
  wenn keine solche Entry vorhanden ist wird *null* zurückgegeben.
- ***size(), isEmpty()***
- ***keySet():*** liefert eine iterierbare Collection mit allen *Schlüsseln*.
- ***values():*** liefert eine iterierbare Collection mit allen *Werten* (Duplikate sind möglich!).
- ***entrySet():*** liefert eine iterierbare Collection mit allen *Entries*.

Beispiel

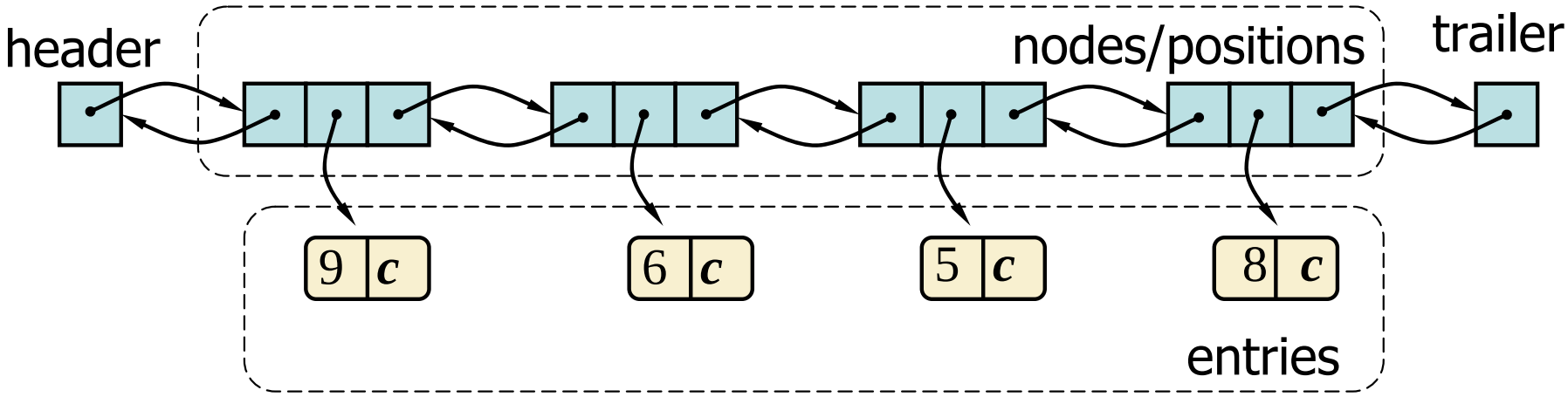
Operation	***Output***	***Map***
isEmpty()	true	{}
put(5,A)	null	{(5,A)}
put(7,B)	null	{(5,A),(7,B)}
put(2,C)	null	{(5,A),(7,B),(2,C)}
put(8,D)	null	{(5,A),(7,B),(2,C),(8,D)}
put(2,E)	C	{(5,A),(7,B),(2,E),(8,D)}
get(7)	B	{(5,A),(7,B),(2,E),(8,D)}
get(4)	null	{(5,A),(7,B),(2,E),(8,D)}
get(2)	E	{(5,A),(7,B),(2,E),(8,D)}
size()	4	{(5,A),(7,B),(2,E),(8,D)}
remove(5)	A	{(7,B),(2,E),(8,D)}
remove(2)	E	{(7,B),(8,D)}
get(2)	null	{(7,B),(8,D)}
isEmpty()	false	{(7,B),(8,D)}
entrySet()	{(7,B),(8,D)}	{(7,B),(8,D)}
keySet()	{7,8}	{(7,B),(8,D)}
values()	{B,D}	{(7,B),(8,D)}

(Annahme: Implementation als unsortierte Liste)

Listen-basierte Implementierung

Mithilfe einer unsortierten Liste kann ein *Map ADT* einfach implementiert werden.

- Die *Entries* der Map können in einer *Liste S* (einer doppelt verketteten Liste) in beliebiger Reihenfolge abspeichert werde.



Der *get(k)* Algorithmus

```
Algorithmus get(k):

B = S.positions()      { B ist Iterator der Positions in S }
while B.hasNext() do
    p = B.next()       { nächste Position in S }
    if p.element().key() = k then
        return p.element().value()
return null            { es gibt keine Entry zum Schlüssel k }
```

Der *put(k,v)* Algorithmus

```
Algorithmus put(k,v):
B = S.positions()
while B.hasNext() do
    p = B.next()
    if p.element().key() = k  then
        t = p.element().value()
        S.set(p,(k,v))
        return t    {Rückgabe des alten Wertes}
S.addLast((k,v))
n = n + 1           {n speichert die Anzahl gespeicherte Einträge}
return null         {es gab keinen Eintrag mit Schlüssel k}
```

Der *remove(k)* Algorithmus

```
Algorithmus remove(k):

B = S.positions()
while B.hasNext() do
    p = B.next()
    if p.element().key() = k  then
        t = p.element().value()
        S.remove(p)
        n = n – 1          {Anzahl Einträge nimmt um 1 ab}
        return t           {Rückgabe des entfernten Werts}
return null                {es gibt keinen Eintrag zum Schlüssel k}
```

Analyse der List-basierten Map

Performance:

- ***put()*** benötigt $O(n)$:
  Ein neuer Eintrag kann am Anfang oder Ende der Sequenz einfügt werden, also $O(1)$, aber der Algorithmus geht mit $O(n)$, weil er dazu zuerst die ganze Liste durchsuchen muss.
- ***get()*** und ***remove()*** benötigen $O(n)$:
  Im schlimmsten Fall muss die ganze Sequenz durchsucht werden.

⇨ Die Implementierung mithilfe einer unsortierten Liste ist nur bei kleinen Maps sinnvoll.

Sets, Multisets & Multimaps

Set:

- Collection von *Elementen*, *ohne Duplikate*.
  - *Elemente* eines *Sets* entsprechen den *Keys* einer *Map*, aber ohne zugehörige *Values*.

Multiset:

- Ein *Multiset* entspricht einem *Set*, wobei aber *Duplikate erlaubt* sind.

Multimap:

- Eine *Multimap* entspricht einer *Map*, wobei es zu einem Key *mehrere Values* geben kann.

Set ADT

add(e): Adds the element e to S (if not already present).

remove(e): Removes the element e from S (if it is present).

contains(e): Returns whether e is an element of S.

iterator( ): Returns an iterator of the elements of S.

There is also support for the traditional mathematical set operations of ***union***, ***intersection***, and ***subtraction*** of two sets S and T:

$$\begin{aligned} S \cup T &= \{e\colon\ e \text{ is in } S \text{ or } e \text{ is in } T\}, \\ S \cap T &= \{e\colon\ e \text{ is in } S \text{ and } e \text{ is in } T\}, \\ S - T &= \{e\colon\ e \text{ is in } S \text{ and } e \text{ is not in } T\}. \end{aligned}$$

addAll(T): Updates S to also include all elements of set T, effectively replacing S by $S \cup T$.

retainAll(T): Updates S so that it only keeps those elements that are also elements of set T, effectively replacing S by $S \cap T$.

removeAll(T): Updates S by removing any of its elements that also occur in set T, effectively replacing S by $S - T$.

Multisets

- *Multisets* erlauben die Speicherung von *Duplikaten* (Mehrfacheinträgen).

- Beispiel:
  - { a, ***b***, ***b***, c, d, e, ***f***, ***f*** }

- Alternativer Begriff für *Multiset* : *Bag*

Multimaps

- Ein *Multimap* ist ähnlich aufgebaut wie eine *Map*, speichert also *<k,v>-Paare* als *Entries*.
- Im Gegensatz zu *Maps* kann ein *Multimap* aber zu einem *Schlüssel k mehrere Einträge* abspeichern, analog zu einem Wörterbuch:
  - *<**k**,v1>* und *<**k**,v2>*

- Beispiel:
  - *Schloss Gebäude*
  - *Schloss Verriegelung*

- Alternativer Begriff für *Multimap* : *Dictionary*

Lösungsansatz:

- Ansatz 1: *Schlüssel k* verweist auf eine *Collection mit Werten zu k* .
- Ansatz 2: anpassen der darunterliegenden Datenstruktur.

Multimap ADT

get(k):	Returns a collection of all values associated with key k in the multimap.
put(k, v):	Adds a new entry to the multimap associating key k with value v, without overwriting any existing mappings for key k.
remove(k, v):	Removes an entry mapping key k to value v from the multimap (if one exists).
removeAll(k):	Removes all entries having key equal to k from the multimap.
size( ):	Returns the number of entries of the multiset (including multiple associations).
entries( ):	Returns a collection of all entries in the multimap.
keys( ):	Returns a collection of keys for all entries in the multimap (including duplicates for keys with multiple bindings).
keySet( ):	Returns a nonduplicative collection of keys in the multimap.
values( ):	Returns a collection of values for all entries in the multimap.

Hash-Table

Eine ***Hash-Table*** (*Hash-Tabelle, Streuspeicher*) ist eine Implementation einer *Map*.

Die Map ADT Methoden (als Repetition):

- *v* ***get****(k)*
- *v* ***put****(k, v)*
- *v* ***remove****(k)*
- *int* ***size****()*
- *boolean* ***isEmpty****()*
- *Set<k>* ***keySet****()*
- *Collection<v>* ***values****()*
- *Set<Entry<k,v>>* ***entrySet****()*

Hash-Funktion und Hash-Tabelle

Eine *Hash-Funktion* ***h*** bildet einen *Key* auf einen Integer in einem fixen Interval [0, ***N*** - 1] ab.

Beispiel:

$$h(x) = x \bmod N$$

Der Integer ***h(x)*** wird als *Hash-Wert* oder *Hash-Code* des *Key* ***x*** bezeichnet.

Eine *Hash-Tabelle* besteht aus:

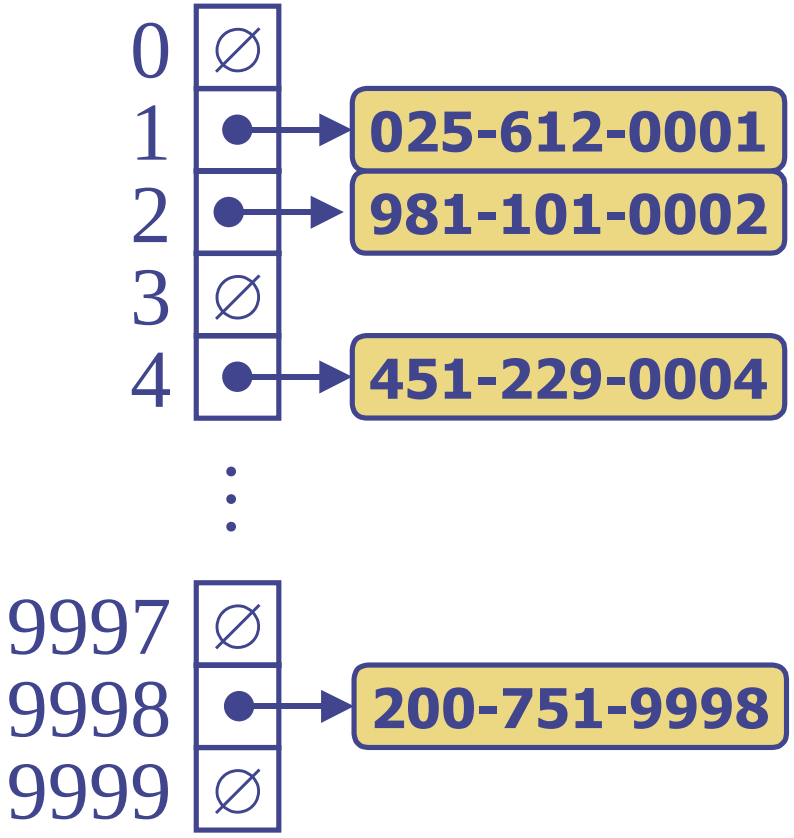
- *Hash-Funktion* ***h***
- *Array* (genannt *Hash-Tabelle*) der Grösse ***N***

Beim Einfügen in die *Hash-Tabelle* versucht man eine *Entry(**k**,v)* beim Index ***i*** = ***h(k)*** zu speichern.

Beispiel

Adresskartei mit der US-`AHV` Nummer als *Key* und dem Namen als *Value* :

- *Hash-Funktion:* ***h(x)*** = letzte vier Ziffern von *Key*
- *Hash-Tabelle:* Array mit ***N*** = 10,000 möglichen Einträgen



Kollisions-Behandlung

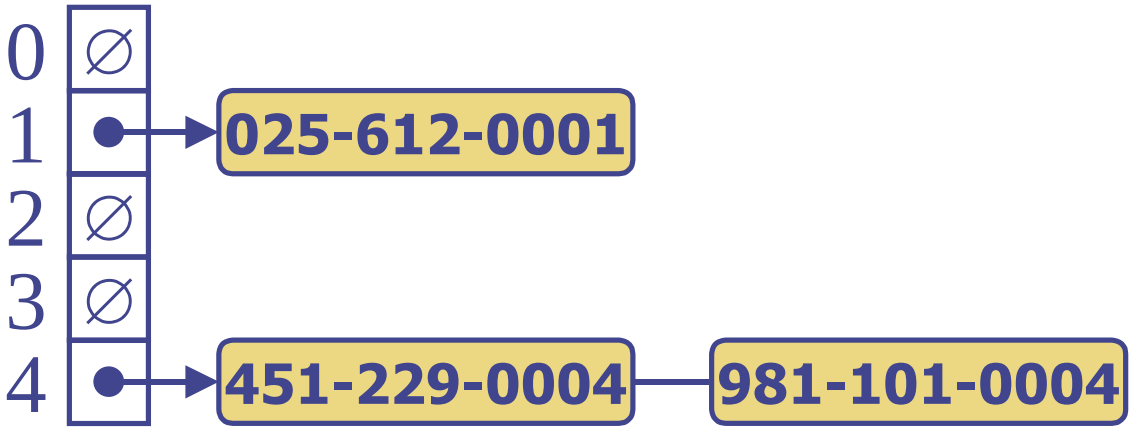
Kollision : für mehrere *Keys* resultieren der gleiche *Index*

Closed-Adressing: Separate Chaining

Eine Zelle der Hash-Table enthält nicht *eine Entry*, sondern eine *Liste von alle Entries*, die mit der Hash-Funktion auf diesen Index gemapped werden.

- *Vorteile:*
  Simple Lösung.

- *Nachteil:*
  Zusätzliche Speicher-Struktur zur Hash-Table nötig.



Open-Adressing

Bei einer Kollision wird ein neuer, nicht kollidierender Index gesucht.

⇨ alle *Entries* werden **in** der Hash-Tabelle gespeichert.

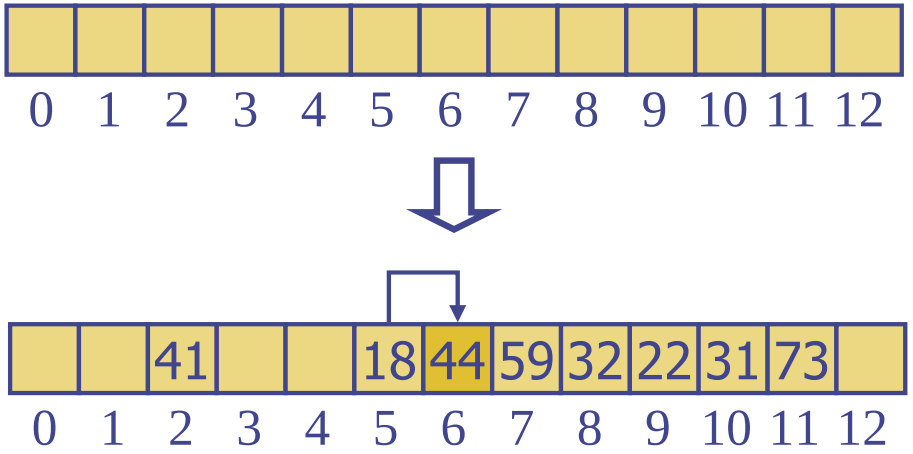
Open-Adressing: Linear-Probing

Bei einer Kollision wird die Entry beim *nächsten leeren Index* (zirkulär) plaziert.

Erweiterungen:

- alternierend vorwärts/rückwärts
- quadratische Sondierung (Schrittweite: AnzahlKollisonen^2)

- Beispiel:
  - $h(x) = x \bmod 13$
  - Einfügen der Keys 18, 41, 22, 44, 59, 32, 31, 73 in dieser Reihenfolge
    18%13=5; 41%13=2; 22%13=9; 44%13=5; 59%13=7…



Open-Adressing: Double Hashing

Bei dem sog. *Double-Hashing* (*doppeltes Hashing*) wird im Falle einer Kollision eine *zweite Hash-Funktion d(k)* für die Bestimmung der Sondierungs-Distanz angewendet.

Beispiel:

$N = 13$

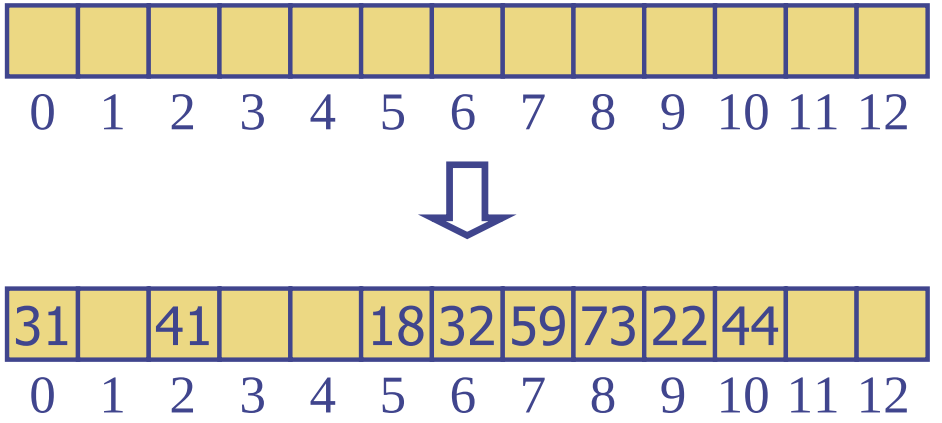
$h(k) = k \bmod 13$

$d(k) = 7 - k \bmod 7$

Einfügen von:

18, 41, 22, 44, 59, 32, 31, 73

k	$h(k)$	$d(k)$	Probes		
18	5	3	5		
41	2	1	2		
22	9	6	9		
44	5	5	5	10	
59	7	4	7		
32	6	3	6		
31	5	4	5	9	0
73	8	4	8		



Performance

Worst case:

- Kollisionen für alle Entries
- ***O(n)*** Zeit für Suchen, Einfügen, Löschen in der Hash-Tabelle

Zeitverhalten ist abhängig vom *Lastfaktor α* = ***Anzahl Entries/N***

Sind die Entries gleichverteilt, so ist der Erwartungswert der Anzahl der Zugriffe beim Einfügen mit offener Adressierung: ***1 / (1 - α)***

Rehash : Neuaufbau der Hash-Tabelle mit grösserer Kapazität (automatisch)

Das *erwartete* Laufzeitverhalten einer Hash-Tabelle ist ***O(1)*** .

In Praxis ist *Hashing* sehr schnell, falls der Lastfaktor nicht zu gross ist.

Anwendungsbeispiel für Hash-Tabellen:

- Kleine Datenbanken
- Compilers
- Browser Caches

Java:
Object.*equals()* und Object.*hashCode()*

Wenn für eine Klasse ***equals()*** überschrieben wird muss auch ***hashCode()*** überschrieben werden, da sich sonst beim Einsatz von *Hash-Maps* des *Collection-Frameworks* Fehler ergeben (Entry wird nicht gefunden).

Javadoc API-Beschreibung von *java.lang.**Object.equals()*** :

*"Note that it is generally necessary to override the hashCode method whenever this method is overridden, so as to maintain the general contract for the hashCode method, which states that **equal objects must have equal hash codes**."*

Python: *__eq__()* und *__hash__()*

Das selbe wie für *Java* gilt auch entsprechend für *Python*.

Für den Test auf Gleichheit zweier Objekte:
__eq__(self, other)

Für die Bestimmung des Hash-Codes eines Objekte:
__hash__(self)